

10. Deep Learning 3

Advanced Methods – RNN and Transformers

Machine learning

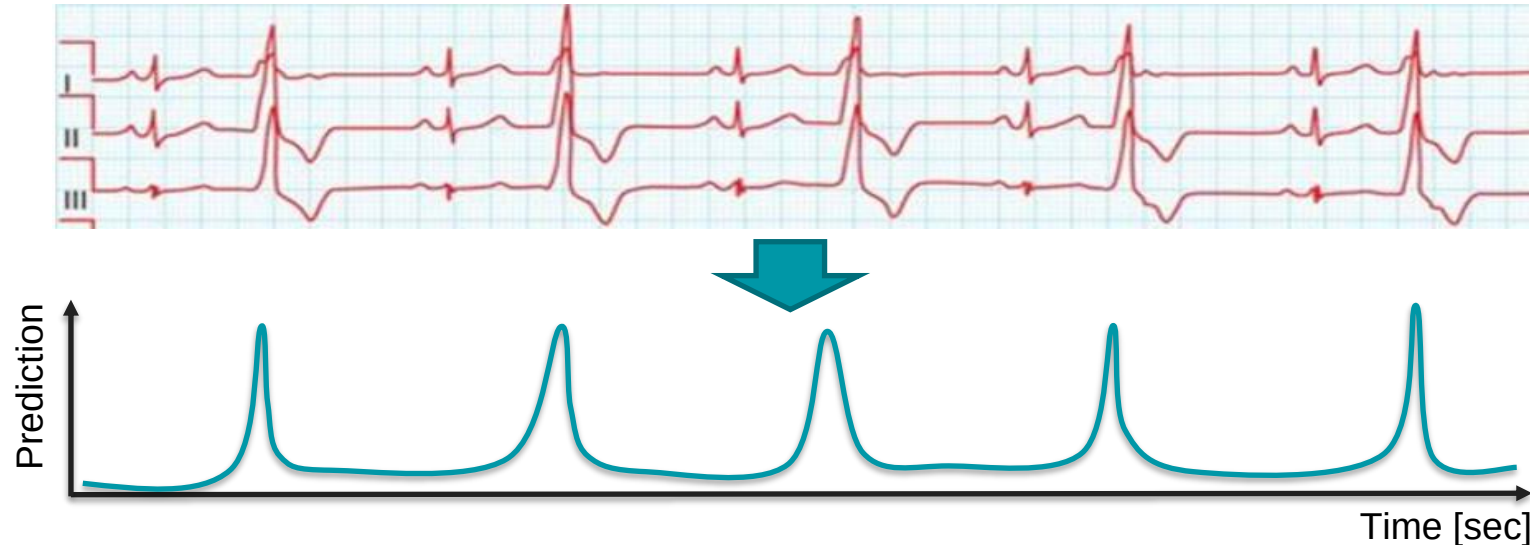
Jiri Chmelik

Department of Biomedical Engineering

Recurrent networks

Motivation

- Analysis (prediction/classification) of data through any dimension (time, space, ...), e.g. prediction of pathologies from a multi-channel electrocardiogram
- This dimension can be e.g. a sequence of words in a sentence (text analysis or generation) or gene sequence
- Moreover, there is a problem that the sequences may be a different length (e.g. ECG signals) => the standard networks have fixed number of input neurons



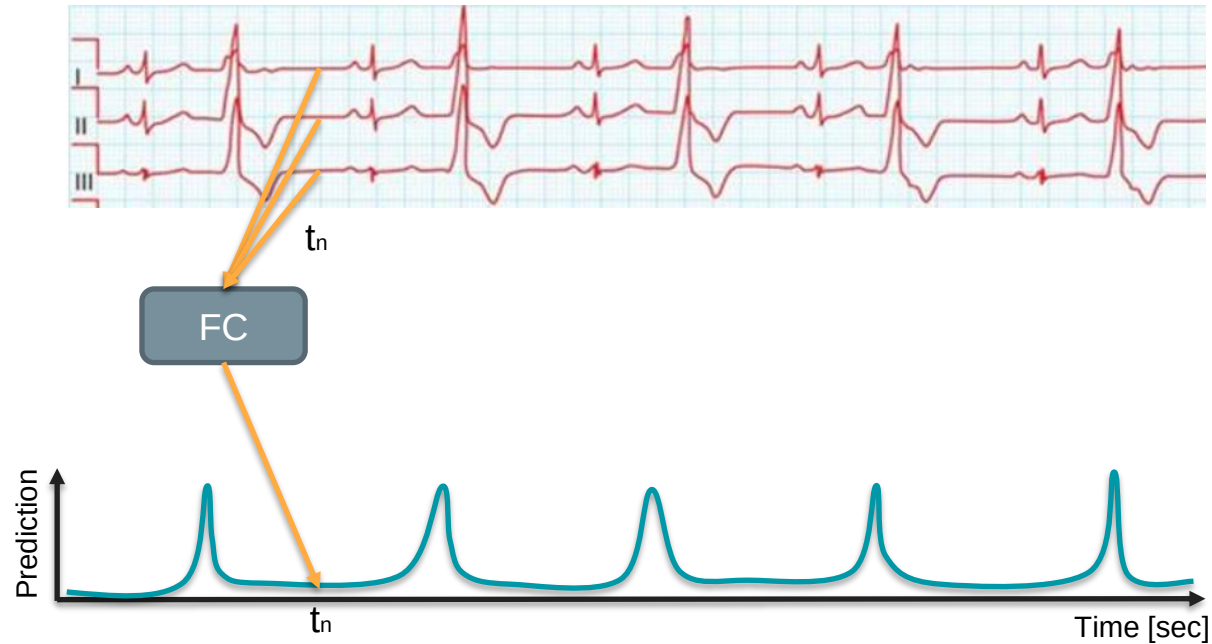
Solution

- Prediction of premature ventricular contractions in each point of dimension (in this case it is time). We can learn a single shallow or deep FC network and predicate “probability” for each time point of the signals.



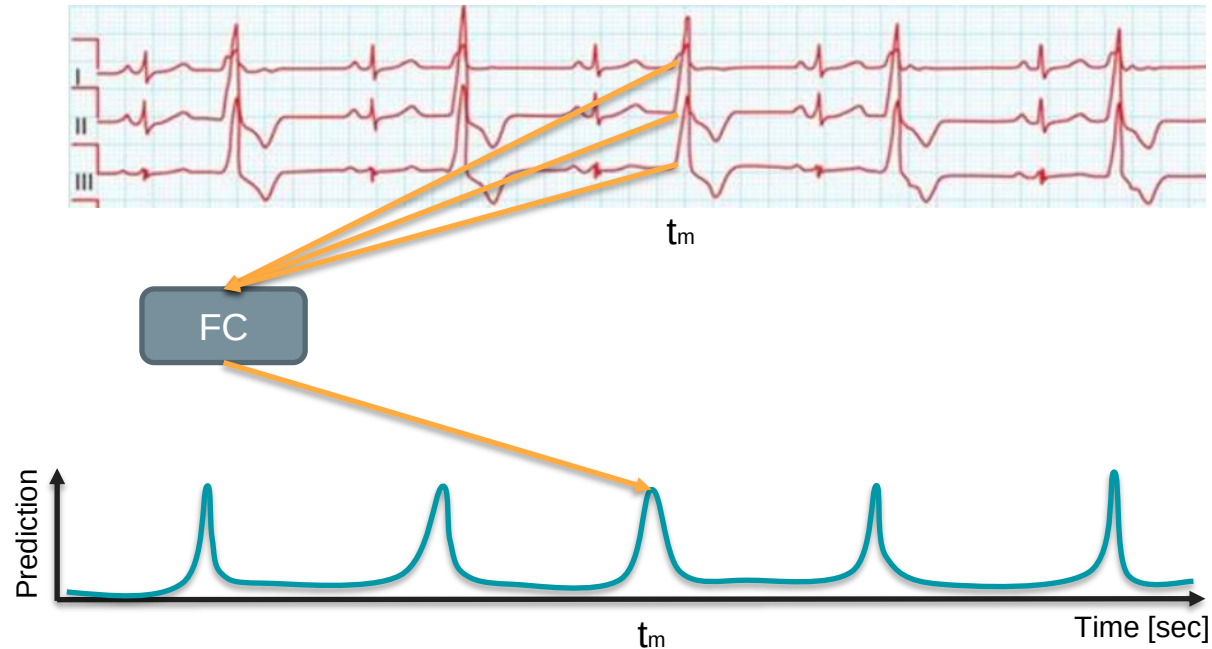
Solution

- Prediction of premature ventricular contractions in each point of dimension (in this case it is time). We can learn a single shallow or deep FC network and predicate “probability” for each time point of the signals.



Solution

- Prediction of premature ventricular contractions in each point of dimension (in this case it is time). We can learn a single shallow or deep FC network and predicate “probability” for each time point of the signals.



Solution

- Prediction of premature ventricular contractions in each point of dimension (in this case it is time). We can learn a single shallow or deep FC network and predicate “probability” for each time point of the signals.
- **But...** we consider **no neighbor time points**, i.e. the prediction is based on only the actual value of signals

Solution

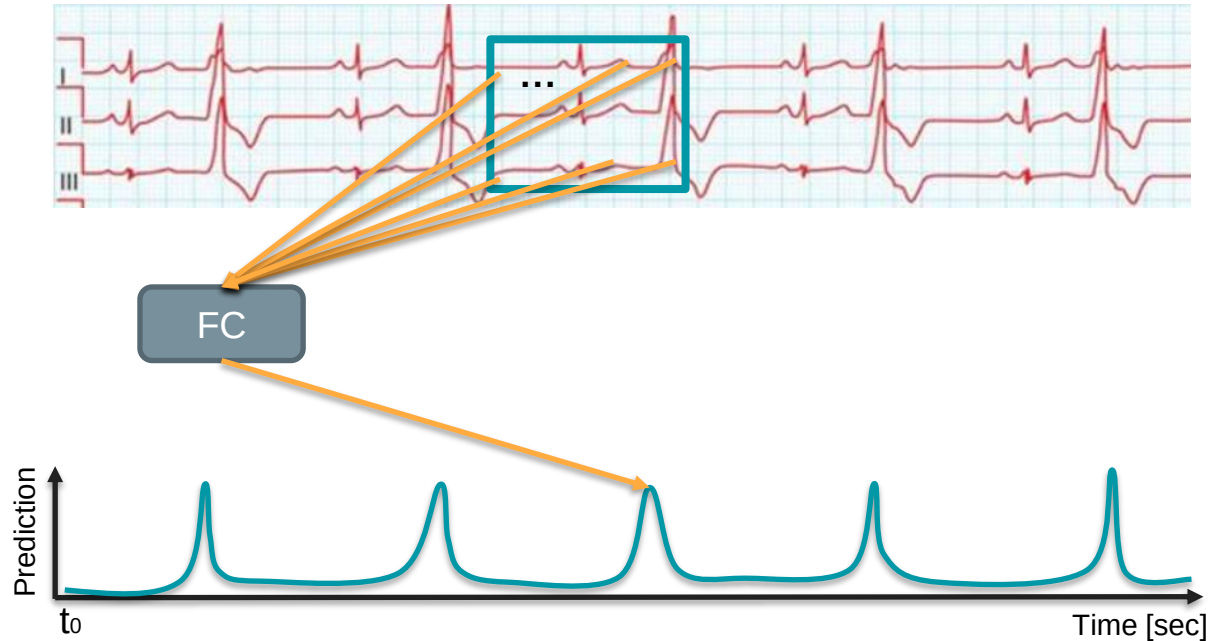
- Extension of number of an input value into the input layer (=> larger number of neurons)
- It is analogous to “sliding window”
- We must define the size of “window”

Usually, we want long time window

Larger window => higher computational complexity

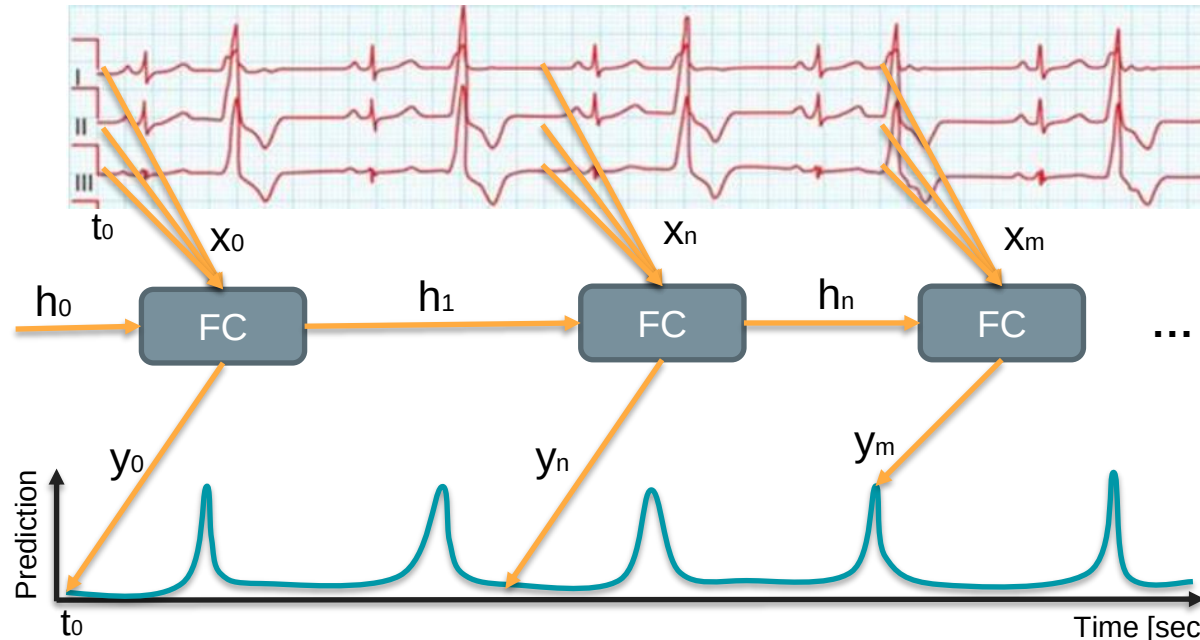
Solution

- Application of the “sliding window”
- Divide the signals into segments (N – length of window) and create a training database
- Number of neurons: $N \times d$ (d ... number of features/signals)



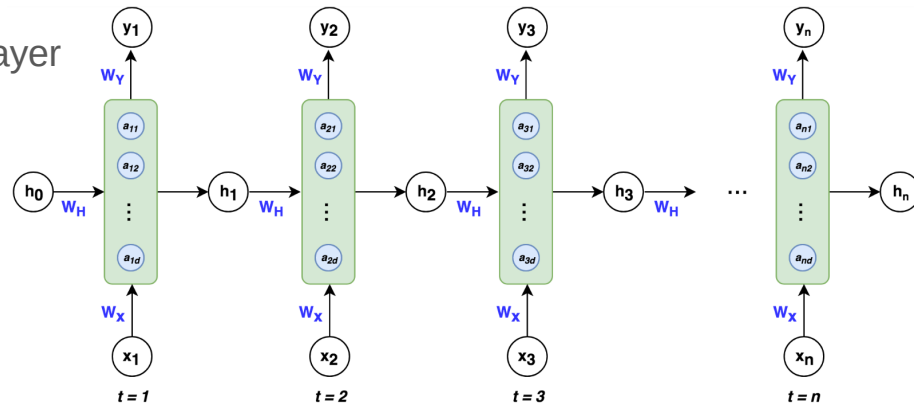
Recurrent Networks

- this network introduces new input ($\mathbf{h}$) vector, which is output from network in previous time point
- FC has thus two input vectors ($\mathbf{h}_{k-1}$ and $\mathbf{x}_{k-1}$) and two outputs ($\mathbf{h}_k$ and $\mathbf{y}_k$)
- for prediction of next time point in signals through any dimension (time), the FC net uses information in form the vector $\mathbf{h}$ from previous time point



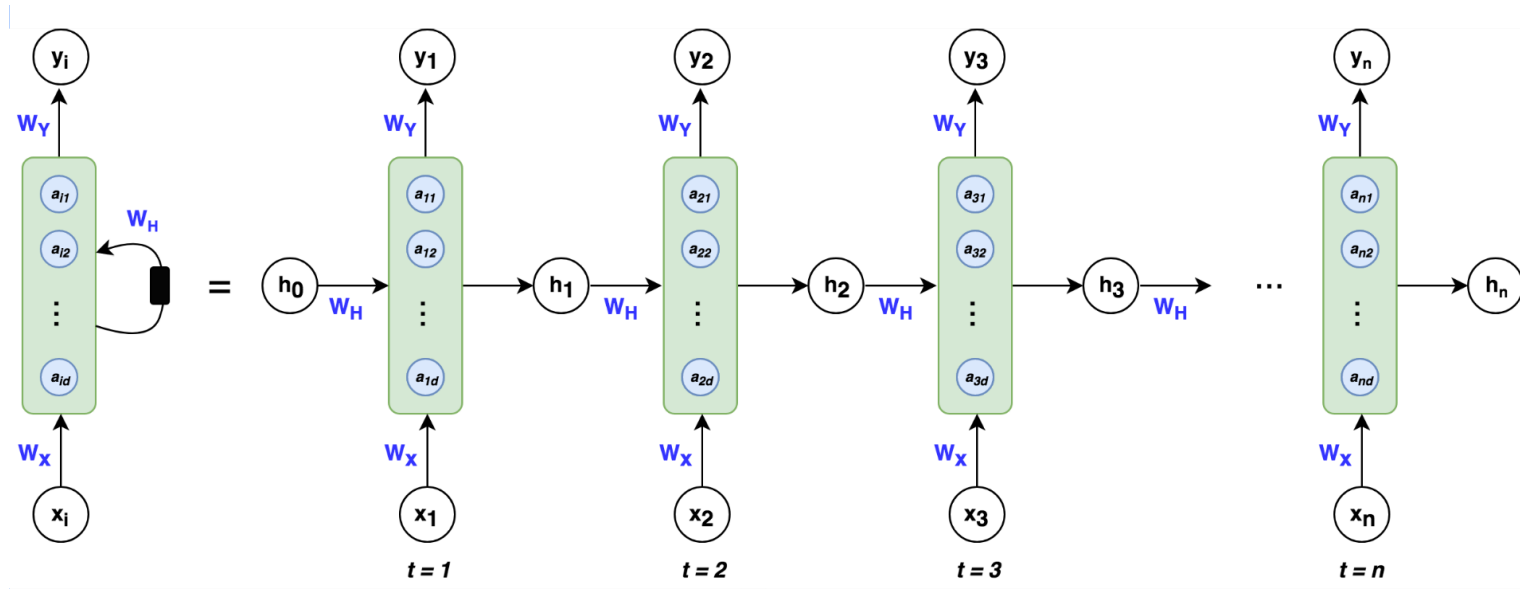
Architecture of Recurrent Networks

- It can be understood as series of FC layers sharing weights
- There are two inputs:
 - $\mathbf{x}_i$ is an input vector of data
 - $\mathbf{h}_{i-1}$ is a hidden layer vector of previous layer
- There are two outputs:
 - $\mathbf{y}_i$ is a vector of predicted values
 - $\mathbf{h}_i$ is a hidden layer vector
- There are three matrix of weights:
 - $\mathbf{W}_x$... weight matrix for transformation of input data data
 - $\mathbf{W}_y$... weight matrix for transformation of vector $\mathbf{h}_i$ to output $\mathbf{y}_i$
 - $\mathbf{W}_h$... weight matrix for transformation of vector $\mathbf{h}_{i-1}$ to output $\mathbf{h}_i$



Architecture of Recurrent Networks

Because the all weight matrices of FC for each “time” are shared, we can use **RECURSIVE APPROACH**



Vanilla recurrent network “Elman’s network”

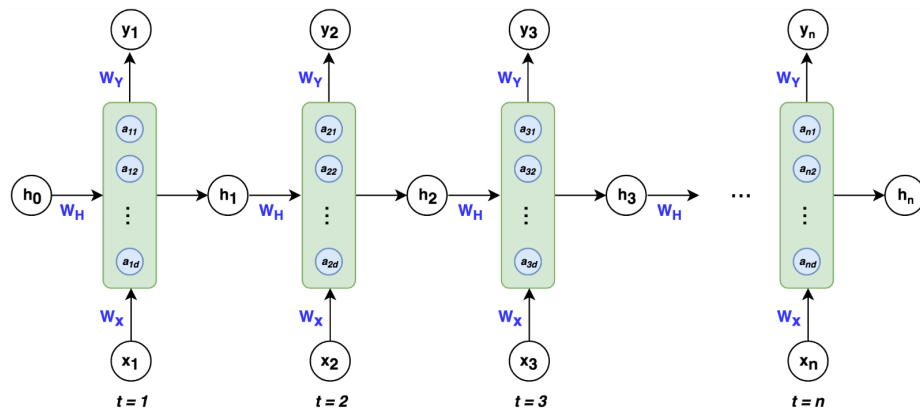
- The value of **hidden layer vector** $\mathbf{h}_t$ can be compute as follows:

$$\mathbf{h}_t = \alpha_h(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\mathbf{y}_t = \alpha_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$

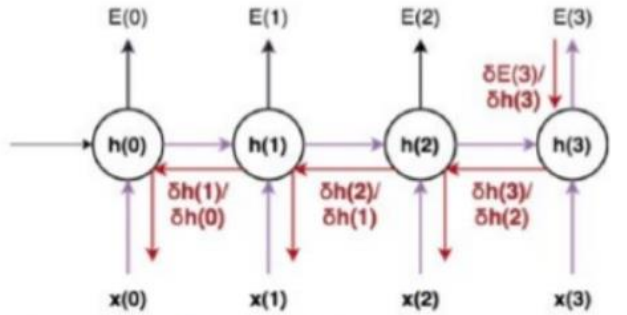
α_h ... activation function for calculation h vector
 α_y ... activation function for calculation y vector

- The networks share weight matrices
- Output is vector $\mathbf{y} = [y_1, \dots, y_n]$ of n predicted values
- Backprop. is based on an average or sum of error over “time”
- Information from older pattern is cumulated, but if we use sigmoid => gradual forgetting

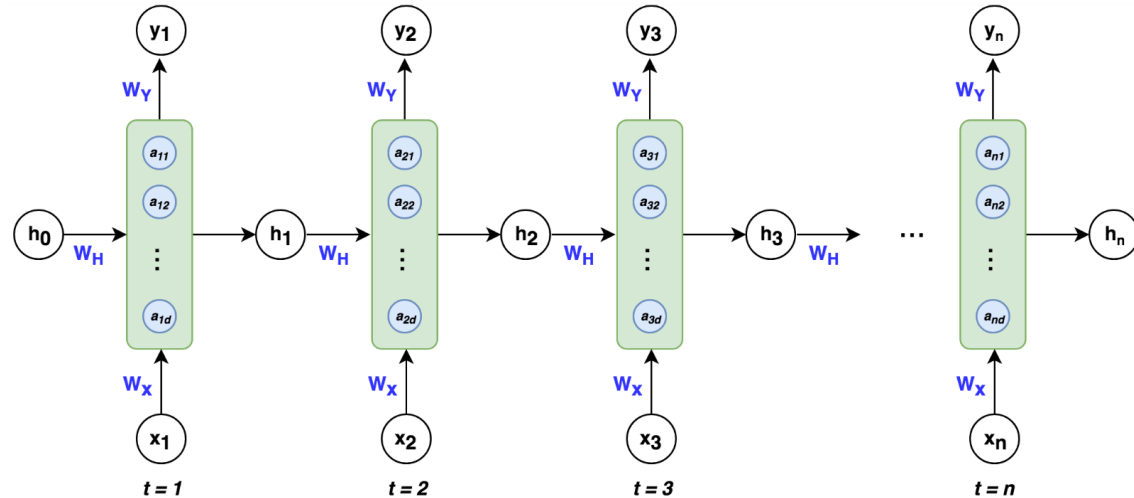


Pipeline

- 1) Set an initial values of weight matrices ($\mathbf{W}_x$, $\mathbf{W}_h$, $\mathbf{W}_y$)
- 2) Predict an output value for each time point $\mathbf{y} = [y_1, \dots, y_n]$
- 3) Compute an average error of prediction
- 4) Modified a weight matrices via backprop.
- 5) Repeat step 2-4



Example back-prop in time with 3 time-steps



Utilizing

- **Natural language processing**
 - obtaining the context from sentence/text
 - classification of text document
 - automatic translation
 - spam detection
- **Speech recognition**
 - transfer spoken word to written text
- **Financial prediction, ...**
- **Analysis of sequences:**
 - gene sequences (e.g. secondary structure prediction, ...)
 - biomedical signals (e.g. prediction of pathological stages, ...)
 - biomedical images (e.g. explain Images, video, ...)

...

Summary

Pros

- predictor, which uses older prediction in the form activation values (h_{i-1}) from previous “iteration”
- learned gradual forgetting older prediction
- information transfer over time (or another dimension)
- we can process sequence of different lengths
- if the input size is larger, the model size does not increase

Cons

- difficult training (depend on length of signals)
- for each “time” point, the prediction of networks is required – must be performed iteratively
- gradient vanishing and exploding problems
- cannot be parallelized
- cannot be used transfer learning

There are many modifications of RNNs (e.g., network doesn't use $\mathbf{W}_y$ or bi-directional, ...)

LSTM and GRU

Long short-term memory, Gated recurrent units

Principle of LSTM

- This network introduces into RNN the recurrent “forget gates”, which allow use very long memories.
- The LSTM approach solves vanishing gradient or exploding problem in backprop.
- It can work even given long delay between significant events

$$\mathbf{F}_t = \alpha_f (\mathbf{W}_f [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

$$\mathbf{I}_t = \alpha_i (\mathbf{W}_i [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

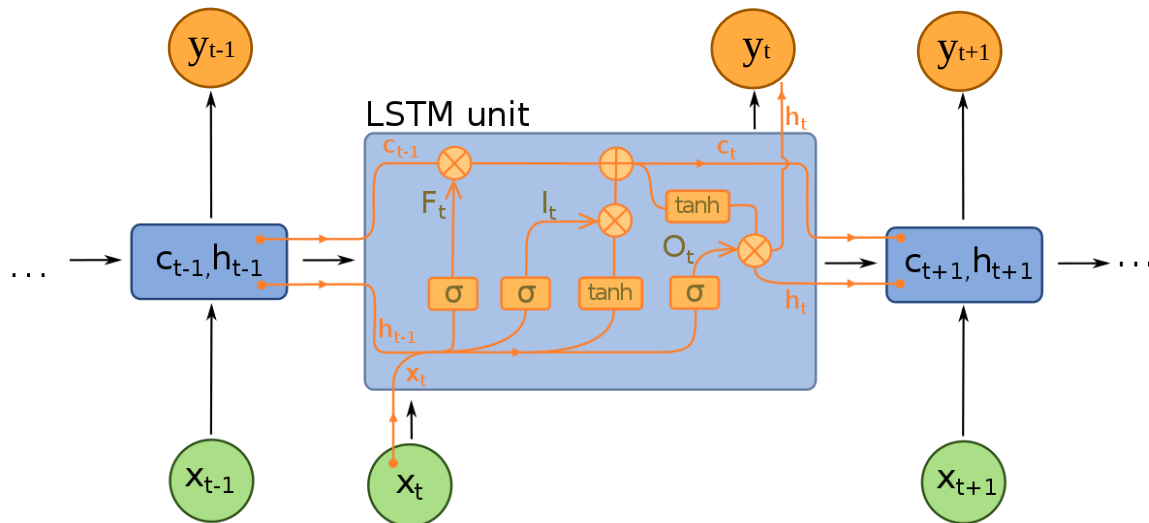
$$\tilde{\mathbf{C}}_t = \alpha_c (\mathbf{W}_c [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

$$\mathbf{C}_t = \mathbf{F}_t \mathbf{C}_{t-1} + \mathbf{I}_t \tilde{\mathbf{C}}_t$$

$$\mathbf{O}_t = \alpha_o (\mathbf{W}_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{O}_t \alpha_h (\mathbf{C}_t)$$

$$\mathbf{y}_t = \mathbf{h}_t$$



Summary

Pros

- very successful modification of RNNs and their utilizing
- solves a vanishing gradient problem (error can be backpropagated through time and layers)
- long memory
- we can process sequence of different lengths

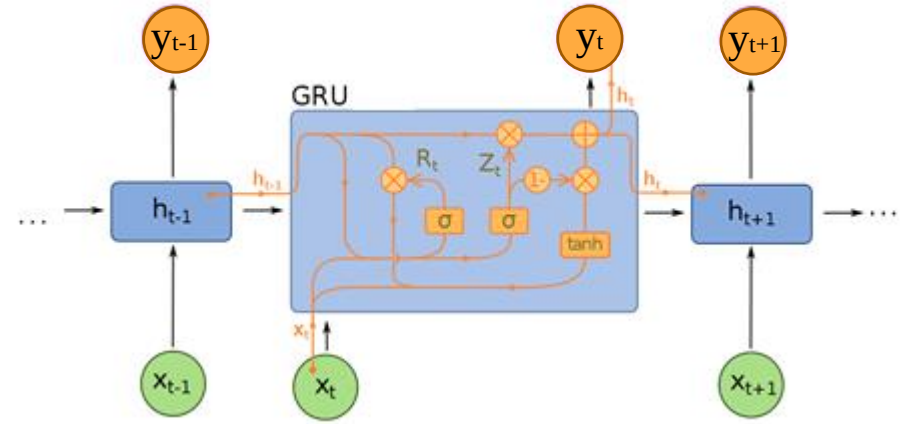
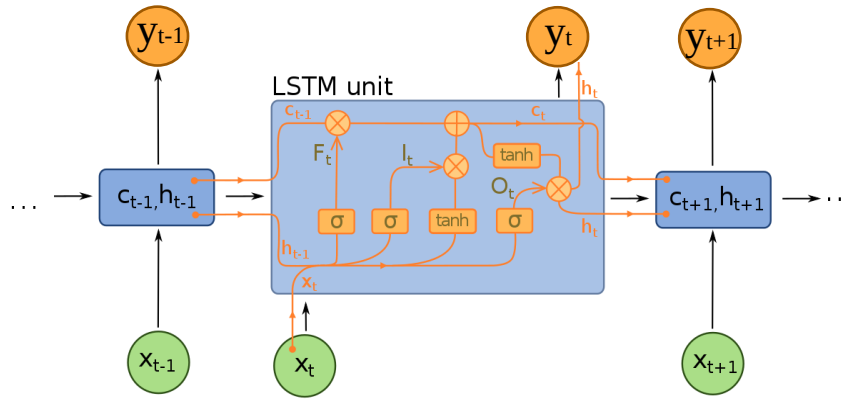
Cons

- uses sigmoid or tanh activation function – more prone to saturation (you can use “softsign”)
- Iterative calculation – high computational complexity
- cannot be parallelized
- cannot be used transfer learning
- cannot be re-trained – generalization problem

There are many modifications of the LSTM (e.g. adding “peephole connection”, Depth Gated RNNs, Clockwork RNNs, grid LSTM or **GRU**)

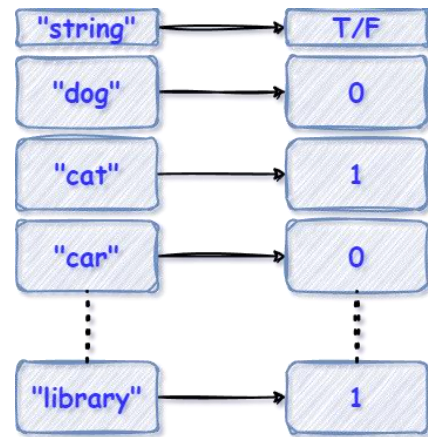
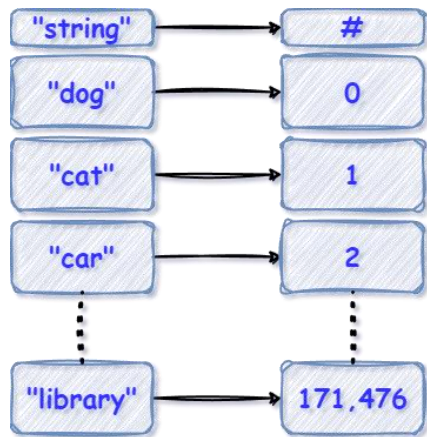
Principle of GRU

- This network introduces Gated Recurrent Units (GRU)
- It is simpler than LSTM and has fewer parameters
- In some cases, it has similar performance like LSTM



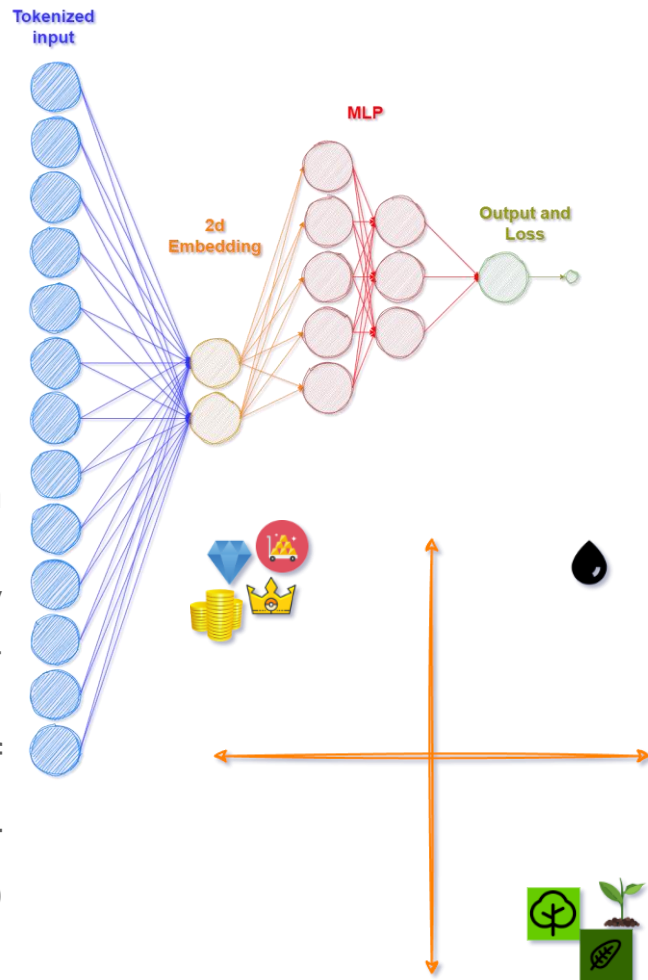
Tokenization and Embeddings

- Tokenization = conversion from original input format (words, images, etc.) into machine readable format (vector of values),
 - Could be simple vocabulary or dictionary (e. g. each unique word has unique number), sparse vector coding (for some word True value) or one-hot-encoded (for one word True value)
 - Highly ineffective (especially with large number of unique words)
 - Missing the meaning, context, similarities or relations between words



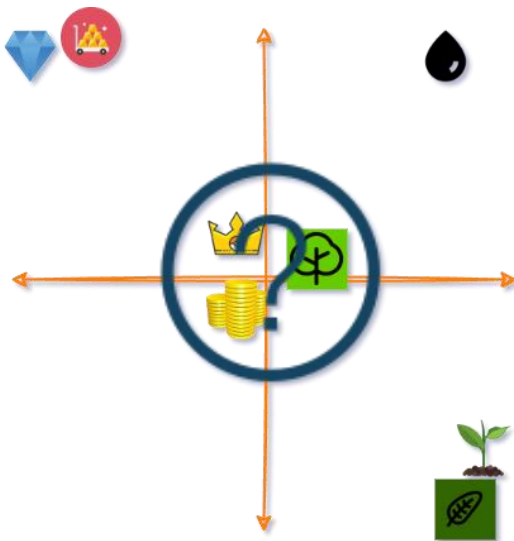
Tokenization and Embeddings

- Embedding = tokens in some d-dimensional space describing hidden relationships between tokens -> similar tokens will be close to each other in the embedding – via mapping $f: X \rightarrow Y$
 - Hundreds or thousands of dimensions (should be much lower than number of unique tokens)
 - Several approaches – most frequently using supervised learned neural networks (e. g. Word2Vec, CBOW Skip-Gram)
- Preservation of information about position of word in sentence via positional encoding – usually simple adding of vector of indices to embedding vector.



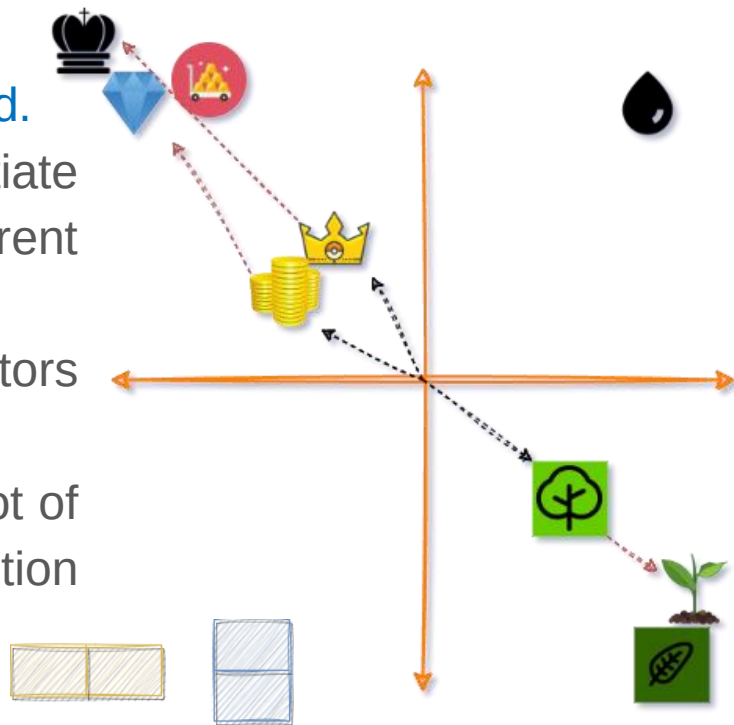
Contextual problem

- The same word could have different meaning!



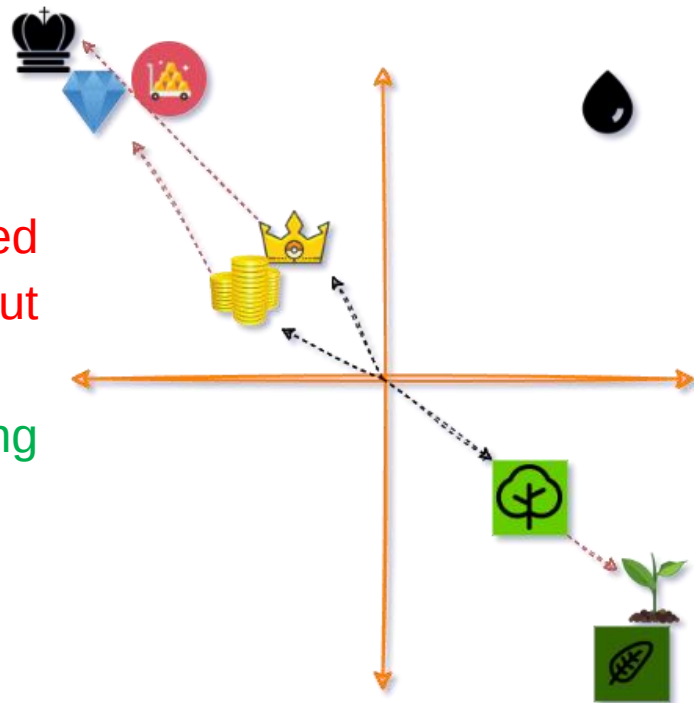
Contextual problem – solution = SELF-ATTENTION

- The **king** put on his crown.
 - A crown of the **tree** is **growing**.
 - A **valuable** Czech Crown has been invested.
 - Using contextual information to differentiate between the same words with different meaning.
1. Calculating the **similarity** between all vectors to each other as their **dot product**,
 2. Scale scores by their division by square root of vector lengths and apply SoftMax function (positive values only).
 3. Shift the positions using the similarities.

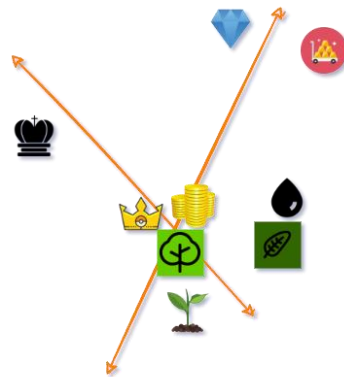


Contextual problem – solution = SELF-ATTENTION

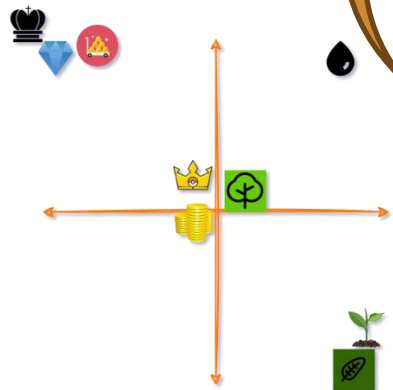
- The **king** put on his crown.
 - A crown of the **tree** is **growing**.
 - A **valuable** Czech Crown has been invested.
- 
- Currency and jewellery is still not distinguished here. One embedding is not enough, But building many of them is too demanding!
 - Solution: to build new embedding by modifying existing one by linear **transformation**.



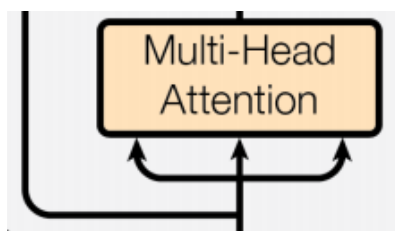
Transformers



Linear transformation of embedding
via matrix multiplication



Transformers

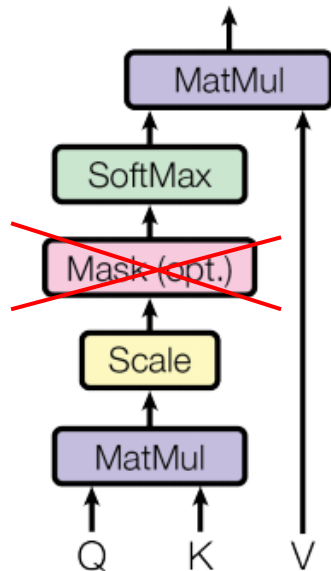


Transformers

$$MHA(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O$$
$$\text{head}_i = A(QW_i^Q, KW_i^K, VW_i^V)$$

Scaled Dot-Product Attention

$$A(Q, K, V) = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Q – query

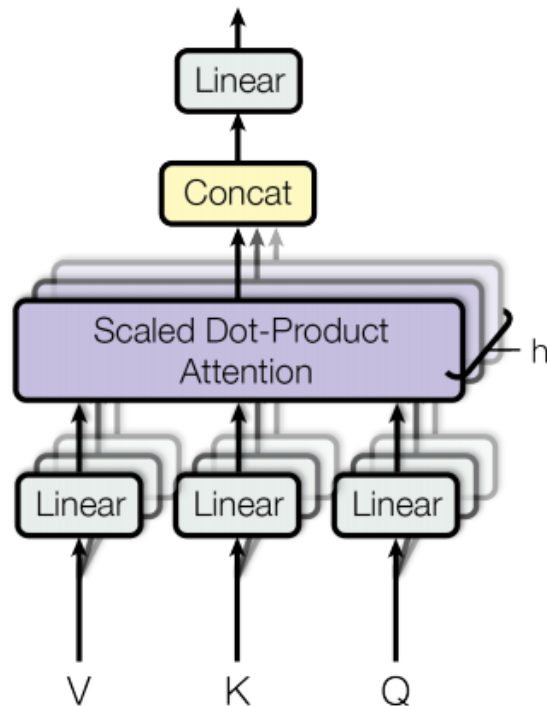
K – key

V – value

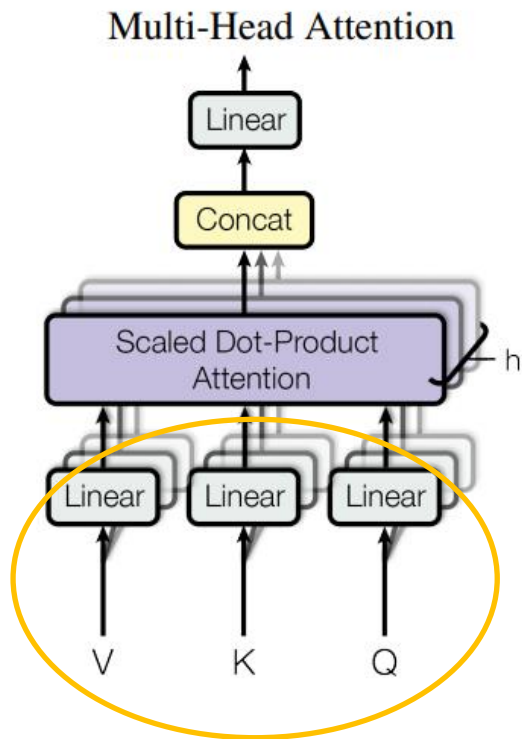
d_k – dimension of key

MatMul – Matrix implementation of Dot Product

Multi-Head Attention



Transformers



Input

Thinking

Machines

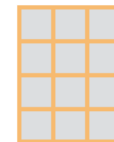
Embedding

 x_1 x_2

Queries

 q_1 q_2  W^Q

Keys

 k_1 k_2  W^K

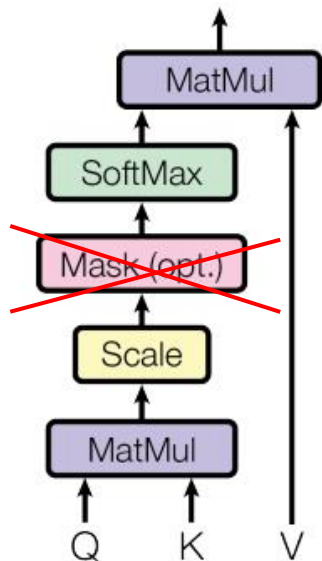
Values

 v_1 v_2  W^V

Multiplying x_1 by the W^Q weight matrix produces q_1 , the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.

Transformers

Scaled Dot-Product Attention



Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Softmax
X
Value

Sum

Thinking

x_1

q_1

k_1

v_1

$q_1 \cdot k_1 = 112$

14

0.88

v_1

z_1

Transformers

Machines

x_2

q_2

k_2

v_2

$q_1 \cdot k_2 = 96$

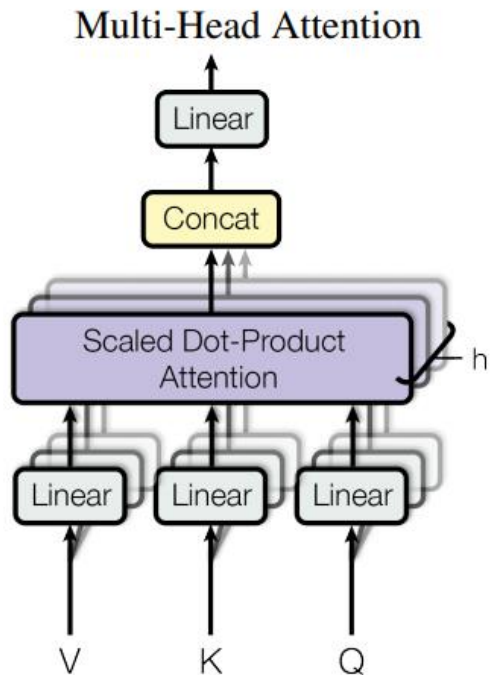
12

0.12

v_2

z_2

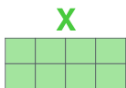
Transformers



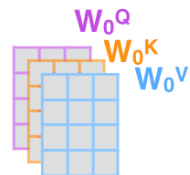
1) This is our
input sentence*

Thinking
Machines

2) We embed
each word*



3) Split into 8 heads.
We multiply X or
 R with weight matrices



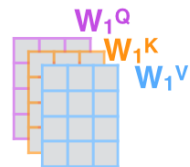
4) Calculate attention
using the resulting
 $Q/K/V$ matrices



5) Concatenate the resulting Z matrices,
then multiply with weight matrix W^O to
produce the output of the layer



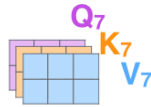
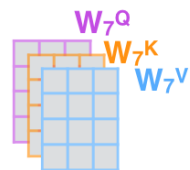
* In all encoders other than #0,
we don't need embedding.
We start directly with the output
of the encoder right below this one



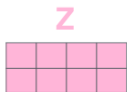
...

...

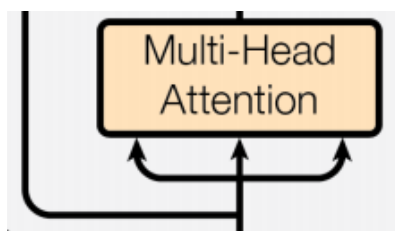
...



W^O



Transformers

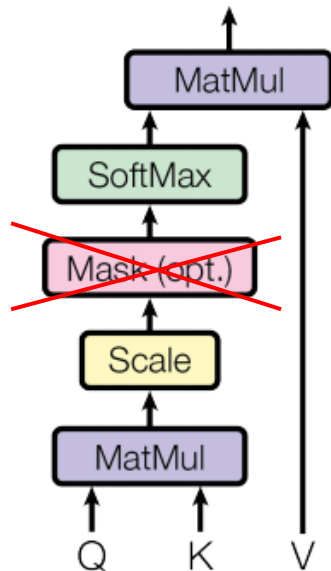


Transformers

$$MHA(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O$$
$$\text{head}_i = A(QW_i^Q, KW_i^K, VW_i^V)$$

Scaled Dot-Product Attention

$$A(Q, K, V) = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Q – query

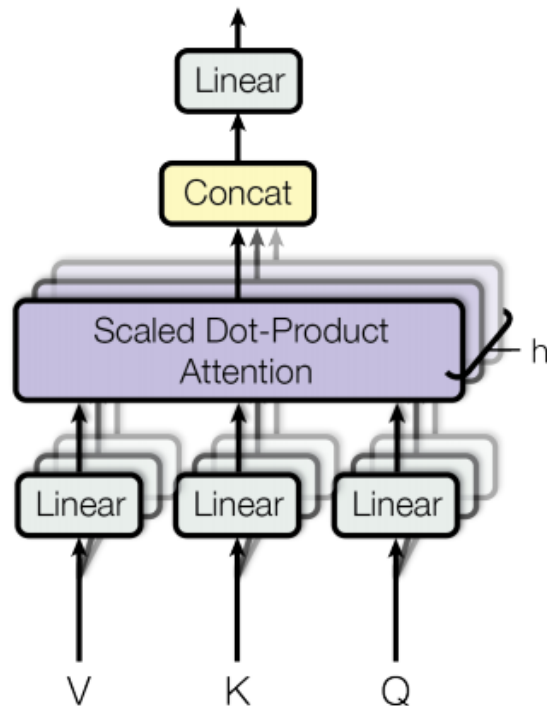
K – key

V – value

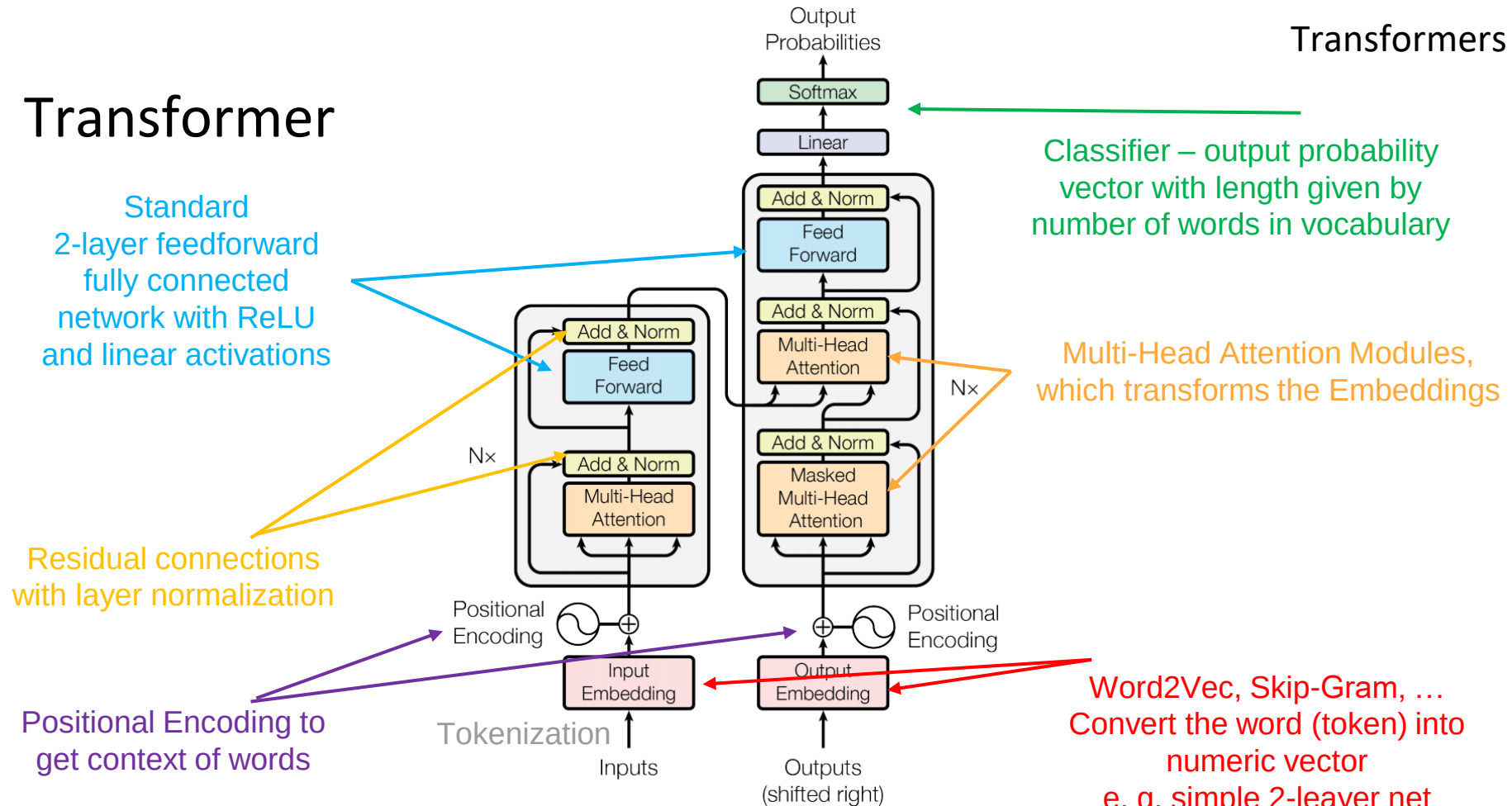
d_k – dimension of key

MatMul – Matrix implementation of Dot Product

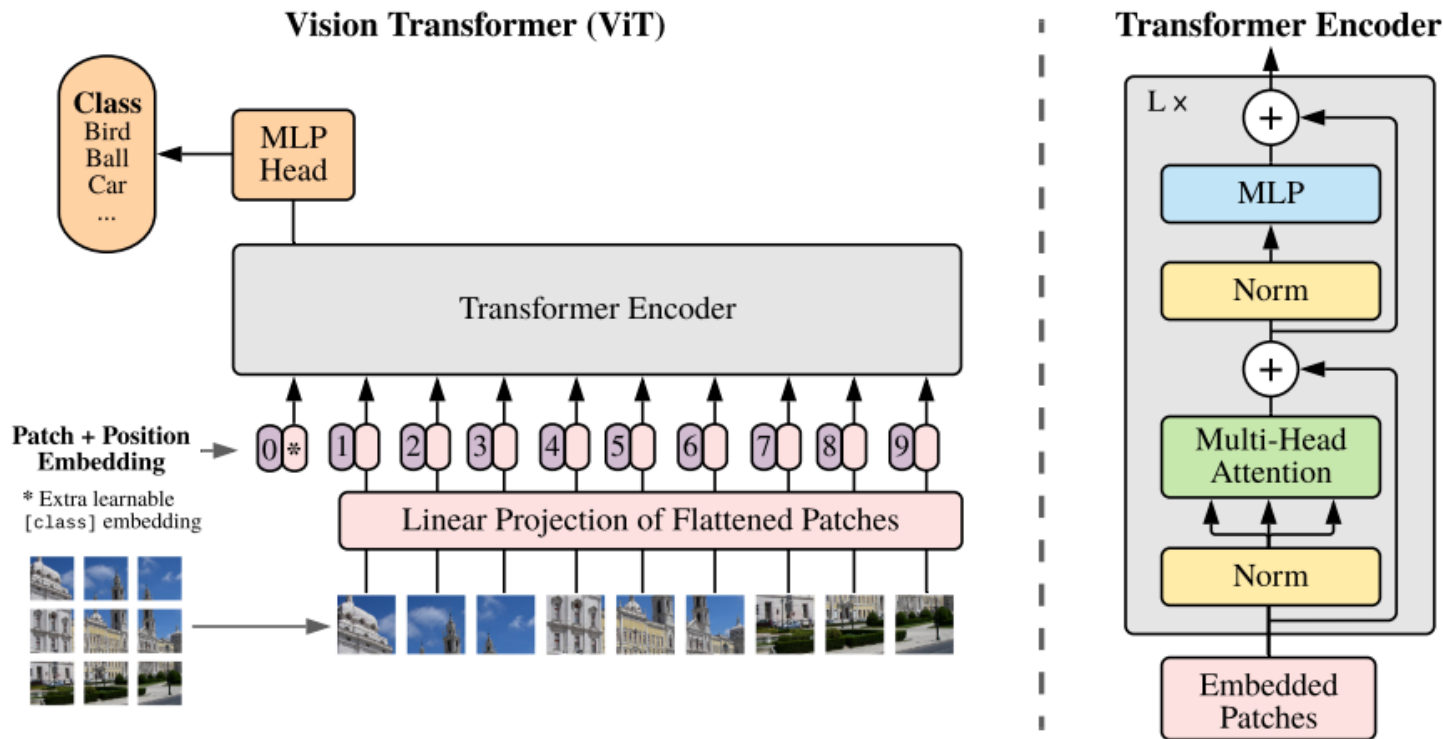
Multi-Head Attention



Transformer



Vision Transformer



Transformers Summary

Properties:

- Huge computational complexity (basic model, 3.5 days on 8 Tesla P100)
- Need of “vocabulary”.
- Questions about very long or infinite signals/sentences/images/sequences.
- Real time running?
- Encoder and decoder have access to all quarry, keys and values and attention just take the important ones – replace of memory in recurrent networks.
- Can be pre-trained and fine-tuned!
- No recurrent block – parallelization is possible!

Pre-trained transformer models:

- NVIDIA Megatron-LM
- OpenAI GPT, GPT-2, GPT-3 (175 billions of params., 355 years on Tesla V100, 4.6M\$), GPT-4
- Google BERT
- Facebook RoBERTa
- Microsoft Turing-NLG
- Transformer-XL, XLNet
- etc.

Recommended sources

- [MIT: RNN, Transformers and Attention](#)
- [Serrano.Academy: Attention](#)
- [Serrano.Academy: Math behind Keys, Queries, and Values matrices](#)
- [Serrano.Academy: Transformers](#)